Constraint-Aware E-Graph Optimization for High-Level Synthesis

Ali Seifeldin
McGill University
Montreal, Canada

Abstract

SEER showed that equality saturation can serve as an optimization substrate for high-level synthesis (HLS), but exposing equivalences is only part of the problem. HLS still needs a way to select implementations under explicit area, latency, and resource constraints. This report presents a datapath-only prototype that extends the SEER idea with constraint-aware extraction. Implemented in Rust with egg, the system saturates arithmetic graphs once, then answers weighted, constrained, and exact area–latency Pareto queries from a bounded frontier extractor backed by a shared analytical cost model over area, latency, DSP/LUT usage, and a normalized power proxy. Evaluation on six arithmetic kernels shows why this extension matters. Default weighted extraction reduces area by 40.9% on average, but regresses latency by 13.7% and normalized power by 44.6% because it often replaces DSP implementations with LUT-heavy alternatives. In contrast, latency-optimal extraction improves latency by 41.7% on average, and constrained queries frequently select qualitatively different implementations from the weighted point. Constraint sensitivity is strongest on `dot16`, where LUT-capped and latency-under-power queries are infeasible. These results support the main claim that, for SEER-style e-graph optimization in HLS, generating equivalent implementations is not sufficient; explicit extraction objectives and budgets are required to select useful hardware mappings.

April 4, 2026

1 Introduction

High-level synthesis raises the abstraction level of hardware design, but arithmetic kernels remain highly sensitive to expression structure, operator binding, and resource allocation [2]. SEER showed that equality saturation is attractive in this setting because an e-graph can retain many equivalent implementations and avoid phase-ordering mistakes during optimization [1]. However, once those alternatives exist, the remaining problem is selection: HLS designers need the fastest implementation under an area budget, the smallest implementation under a latency budget, or a feasible implementation under explicit DSP/LUT limits. An e-graph alone does not answer that question.

This project studies that missing step. The goal is not to reproduce the full SEER stack, but to extend the SEER idea with constraint-aware extraction on a narrower datapath-only platform. The scope is deliberate. By representing kernels directly as arithmetic graphs and using an analytical cost model, the evaluation isolates extraction quality rather than frontend lowering, loop transformation, or memory optimization.

The delivered system therefore makes four contributions:

- a datapath-only SEER-style optimizer with constraint-aware extraction;
- a shared analytical cost model over area, latency, DSP/LUT usage, and power;
- extraction modes for constrained optimization and tradeoff exploration;
- an evaluation showing that constraints materially change the extracted implementation.

The core claim of the report is straightforward: for SEER-style equality saturation in HLS, generating equivalent implementations is not enough. A useful system also needs extraction interfaces that can express budgets, expose tradeoffs, and report infeasibility when constraints cannot be satisfied.

A concrete example makes the selection problem visible. Throughout the report, $A/L/P/D/U$ denotes area, latency, power, DSP count, and LUT count. On `fir8`, the retained frontier contains a low-area weighted point with metrics 25/18/33/0/41, a low-latency point with metrics 38/9/31/10/10, and a lower-power point with metrics 33/16/21/4/9. All three implementations are algebraically equivalent, but they answer different hardware questions. The point of the project is therefore not simply to generate more equivalent expressions than SEER, but to make those expressions selectable under explicit hardware goals.

2 Related Work

egg made equality saturation practical by combining efficient e-matching, rebuilding, and extraction with a usable systems interface [6]. That infrastructure is the mechanism behind this project: rewrites create a space of equivalent arithmetic implementations, and extraction selects one of them.

SEER is the direct conceptual parent on the HLS side [1]. It showed that e-graphs can organize optimization in an

McGill HLS Project Report, , Montreal, Canada
2026.

HLS-relevant compiler flow, but its main contribution is demonstrating equality saturation as an optimization substrate rather than studying constrained selection as a primary object of evaluation. This project takes the next step in a narrower setting: once the e-graph exists, how should an HLS tool extract implementations under explicit budgets and competing objectives?

Datapath-oriented work by Coward et al. is the closest extraction-focused comparison. Their systems showed that e-graphs can recover strong arithmetic datapath optimizations, then extended that line with constraint-aware extraction and power-aware rewriting [3–5]. The present project is smaller in scope and uses an intentionally simple analytical model, but it reaches a similar methodological point: arithmetic equivalences are explored once, then filtered by explicit objectives and budgets rather than by a single scalar heuristic. Relative to SEER, this project contributes constrained extraction and tradeoff analysis, but on a narrower datapath-only experimental platform. It is not an MLIR/CIRCT extension of SEER, not a full HLS frontend, and not a synthesis-calibrated hardware model.

3 Approach and System Design

IR and rewrite space. The IR contains `input`, `const`, `add`, `sub`, and `mul` nodes plus architecture-aware variants such as `add_lut`, `add_dsp`, `mul_lut`, `mul_dsp`, and `mac_dsp`. Rewrites cover constant folding, neutral-element elimination, algebraic reassociation, and datapath mapping alternatives. Additions and subtractions may bind to LUT or DSP implementations; multiplications may bind to LUT or DSP variants; and multiply-accumulate structure can be recognized and replaced with a fused DSP MAC.

Equality saturation and frontier construction. Each benchmark is serialized as a JSON arithmetic DAG, then saturated once in the Rust core using `egg`. The system does not keep one extracted term per e-class, nor does it sample 128 random implementations. Instead, it computes a bounded deterministic frontier. Child frontiers are combined bottom-up, candidates with identical metric vectors are deduplicated, dominated points are removed, and the surviving candidates are retained in a lexicographic order with a 128-candidate cap per e-class and a 16-pass convergence limit. The important design choice is that equality saturation creates the design space once, while extraction reuses that retained frontier to answer multiple hardware questions afterward.

Constraint-aware extraction. Each candidate carries a metric vector (A, L, P, D, U) for area, latency, power, DSP count, and LUT count. Weighted extraction provides a default scalar baseline via $\text{score}(x) = w_a A(x) + w_l L(x) + w_p P(x)$, but the main interface is constrained extraction. The system can minimize latency under an area cap, minimize area under a latency cap, minimize latency under DSP or LUT budgets, and minimize power under a latency constraint. These modes correspond directly to the design questions that motivated the project: fastest under an area budget, smallest under a latency budget, lowest power under a performance target, or feasible under explicit resource ceilings.

Pareto and tradeoff exploration. The project also exposes the design space instead of only returning a single answer. Exact Pareto extraction projects the retained candidate set onto area versus latency to produce a compact non-dominated frontier, and the reporting pipeline uses those points to show where weighted, latency-optimal, and power-aware choices sit relative to one another. This is the main delta from a pure SEER-style "generate equivalences" story: the user can inspect tradeoffs rather than accept one opaque extracted point.

Frontier algorithm. Candidates are pruned using five-dimensional dominance over area, latency, power, DSP count, and LUT count. This is what allows the extractor to preserve non-area information during pruning and still answer different objective queries from the same retained frontier. In other words, the e-graph generates alternatives, while the frontier algorithm preserves enough of them for constrained selection to be meaningful.

Analytical cost model. Metrics are analytical rather than synthesis-derived. Area, power, DSP count, and LUT count are additive over children plus the local operator cost; latency follows the longest dependency path plus local operator latency. Formally, for operation o with children c_i, $A = \sum_i A(c_i) + A(o)$, $P = \sum_i P(c_i) + P(o)$, $D = \sum_i D(c_i) + D(o)$, $U = \sum_i U(c_i) + U(o)$, and $L = \max_i L(c_i) + L(o)$. Power is included to strengthen the constrained-extraction interface, but it is treated as a secondary objective rather than as a claim of device-accurate energy modeling. Table 1 gives the complete operation table used by both the Rust extractor and Python benchmark harness.

4 Implementation

The optimizer core is implemented in Rust 2024 using `egg` 0.10 for equality saturation and `serde`/`serde_json` for the JSON interface. The code path is organized as follows. `ir.rs` defines the request/response schema, metric arithmetic, budget checks, and dominance relation. `optimizer.rs` converts the input DAG to an `egg` expression, runs equality saturation under iteration/node/time limits, constructs the bounded per-eclass frontier, and then answers weighted, constrained, or Pareto queries from the retained root frontier. `main.rs` exposes single-query and batch JSON entry points.

In execution terms, the Rust path has three phases. First, equality saturation builds the shared equivalence space. Second, frontier extraction compresses that space into a bounded

Table 1. Complete operation costs from the shared analytical model.

Op	Area	Lat.	Power	DSP	LUT
`input`	0	0	0	0	0
`const`	0	0	0	0	0
`add`	1	2	1	0	1
`add_lut`	1	2	1	0	1
`add_dsp`	2	1	2	1	0
`sub`	1	2	1	0	1
`sub_lut`	1	2	1	0	1
`sub_dsp`	2	1	2	1	0
`mul`	6	3	3	1	0
`mul_dsp`	6	3	3	1	0
`mul_lut`	4	6	6	0	8
`mac_dsp`	7	3	4	1	0

retained candidate set at the root e-class. Third, query-time selection filters that retained set by budgets and chooses the best feasible candidate for the requested objective. This decomposition matters because the first two phases are shared across all queries for one benchmark, while only the last phase changes when the user asks a different hardware question.

The benchmark harness, figure exporter, and report generator are implemented in Python 3.13.9. Python builds the arithmetic benchmark graphs, computes original baseline metrics from the same shared cost model, assembles batched query sets, invokes the Rust binary once per benchmark, and then flattens the results into CSV, HTML, and paper-ready figures. Batch mode is important because saturation and frontier construction are shared across all queries for one benchmark; only the final selector differs between weighted, constrained, and Pareto extraction. A single `cost_model.json` file is shared across both sides so that baseline measurements, extracted measurements, CSV summaries, HTML output, and paper figures all use identical operator costs.

5 Experimental Setup

The analytical experiments were run on a MacBook Pro with an Apple M4 CPU (10 cores: 4 performance and 6 efficiency) and 16 GB memory, running macOS 26.3.1 on arm64. The evaluation used `rustc` 1.94.0, `cargo` 1.94.0, and Python 3.13.9. A follow-up validation pass used Intel HLS Compiler 21.1 targeting Arria 10 component generation. That pass produced HLS resource estimates and RTL, not full Quartus fitter, timing, or power results; the primary optimization results therefore remain analytical.

The benchmark suite contains six arithmetic kernels chosen to reflect the proposal's datapath focus: an 8-tap FIR accumulation (`fir8`), a 16-element dot product (`dot16`), two GEMM-style accumulations (`gemm_k8` and `gemm_blocked_k8`), a single `conv3x3` window, and a 5-point stencil (`stencil5`). These kernels are represented directly as arithmetic graphs,

Table 2. Benchmark baselines and hard-budget ceilings. Metrics use $A/L/P/D/U$ for area, latency, power, DSP count, and LUT count. Area, latency, and power caps are the original baseline values; the DSP cap is the zero-DSP stress query; the LUT cap is the benchmark-specific resource ceiling.

Benchmark	Kernel	Mul/Add	Base A/L/P/D/U	Caps A/L/P/D/U	Pareto
`fir8`	8-tap FIR	8/7	55/17/31/8/7	55/17/31/0/7	10
`dot16`	dot product	16/15	111/33/63/16/15	111/33/63/0/15	16
`gemm_k8`	GEMM accum.	8/7	55/17/31/8/7	55/17/31/0/7	11
`gemm_blocked_k8`	blocked GEMM	8/7	55/11/31/8/7	55/11/31/0/7	8
`conv3x3`	conv. window	9/8	62/19/35/9/8	62/19/35/0/8	10
`stencil5`	stencil update	5/4	34/11/19/5/4	34/11/19/0/4	8

which keeps the experiment focused on extraction rather than parsing or frontend lowering.

Saturation limits were fixed to 12 iterations, 12,000 e-nodes, and 800 ms per benchmark. For each benchmark, the harness executes one default weighted extraction, one unconstrained latency-minimizing query, one unconstrained power-oriented query, and four proposal-style constrained queries: latency-under-area, area-under-latency, latency-under-power, and power-under-latency. It also runs a zero-DSP query, a LUT-capped query, and an exact area–latency Pareto extraction. Because the evaluation is deterministic and analytical, one run per configuration is sufficient: there is no randomized search or measurement noise to average out. The original arithmetic graph serves as the baseline, default weighted extraction serves as the SEER-style single-answer comparison point, and the constrained modes test whether explicit extraction interfaces materially change the selected design.

The budget profiles are chosen to make the constrained queries interpretable against the original design rather than against an arbitrary external target. Table 2 records these baseline ceilings explicitly. We write each constrained query as objective | hard constraint. 'min Lat | Area cap' uses the original area as a hard ceiling and therefore asks whether latency can be improved without growing the implementation. 'min Area | Lat cap' and 'min Power | Lat cap' use the original latency as a ceiling and therefore ask whether the datapath can be compacted or de-powered without slowing it down. 'min Lat | Power cap' uses the original power proxy as a ceiling and asks whether the same power class can support a faster mapping. The remaining two profiles intentionally stress resource substitution: 'min Lat | DSP=0' forces a LUT-only implementation, while the benchmark-specific LUT caps expose whether a kernel can escape LUT-heavy weighted solutions at all.

We also instrumented the Rust implementation to separate equality-saturation time, frontier-construction time, and query-selection time on synthetic FIR and dot-product size sweeps. Those measurements are useful for explaining the pipeline, but they are treated as supplementary diagnostics rather than as a main claim because absolute runtime is

machine-dependent and small relative to the analytical optimization story developed here.

6 Evaluation

Across the suite, weighted extraction reduces analytical area by 40.9% on average but regresses latency by 13.7% and normalized power by 44.6%. Latency-optimal extraction instead improves latency by 41.7% on average, while power-optimal extraction is conservative and improves normalized power by 7.8%. These averages separate the roles of the modes: weighted extraction is primarily an area heuristic, latency-optimal extraction is a performance-seeking mode, and power-optimal extraction is a secondary objective. The rest of the evaluation asks whether changing the selector changes the implementation qualitatively and whether infeasible cases line up with the intended resource restrictions.

Q1: Is default weighted extraction sufficient? Table 3 shows that the answer is no. Default weighted extraction reduces area on every benchmark and removes DSP use aggressively; the weighted point uses zero DSPs on all six benchmarks while often increasing LUT count substantially. It improves latency and power together only on `conv3x3`. Across the suite it reduces area by 40.9% on average while regressing latency by 13.7% and normalized power by 44.6%. The pattern is consistent with the cost model: the weighted point often trades DSP implementations for LUT-heavy arithmetic, which lowers the scalar area metric but lengthens the critical path and increases the power proxy.

The per-benchmark rows make this failure mode concrete. `fir8` and `dot16` show the classic tension that motivated the project: the weighted point moves strongly downward in area, but not in latency or power. `gemm_k8`, `gemm_blocked_k8`, and `stencil5` are sharper cases, where the weighted point is clearly wrong if latency or power matters. `conv3x3` is the useful exception; it shows that when the rewrite space contains a genuinely better operating point, the framework can still exploit it. **This shows that a SEER-style single extracted point is not a reliable HLS answer once hardware constraints matter.**

Q2: Do explicit constraints and objectives change the chosen implementation? Figure 1 and Table 3 show that the added extraction modes are not cosmetic. On `fir8`, the weighted point, latency-optimal point, and power-aware point are all different; none dominates the others across area, latency, and power simultaneously. The same qualitative separation appears across the suite. Latency-optimal extraction improves latency by 41.7% on average relative to baseline, and exact Pareto extraction still exposes 10.5 nondominated area–latency points per benchmark on average, so the equality-saturated design space is not degenerate under this model. Power remains a secondary story: it helps most clearly on `fir8` and `conv3x3`, but it is less reliable on kernels such as `dot16`, which is consistent with the fact that the power metric is only an analytical proxy. **This shows that the project's main contribution is not merely generating equivalences, but selecting materially different implementations from the same e-graph under different objectives.**

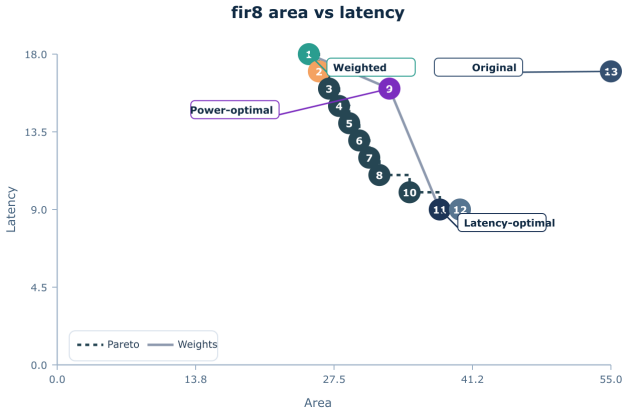

Figure 1. Representative `fir8` area–latency tradeoff. The weighted, latency-optimal, and power-aware points occupy distinct locations on the retained frontier. Their full $A/L/P/D/U$ totals are shown in Table 3.

Constraint Sensitivity Across Benchmarks

Columns are objective | hard constraint; cell text is target-metric improvement vs weighted extraction.

better · same · worse · infeasible

	min Lat | Area cap	min Area | Lat cap	min Lat | Power cap	min Power | Lat cap	min Lat | DSP=0	min Lat | LUT cap
fir8	+50%	-4%	+50%	+36%	+0%	+50%
dot16	+42%	-4%	inf	+19%	+0%	inf
gemm-k8	+50%	-8%	+15%	+44%	+0%	+15%
blocked-k8	+36%	-13%	+21%	+44%	+0%	+50%
conv3x3	+50%	+0%	+50%	+17%	+0%	+50%
stencil5	+43%	-12%	+21%	+44%	+0%	+50%

Figure 2. Constraint sensitivity across the benchmark suite. Column labels use objective | hard constraint. Each cell reports improvement in the constrained query's target metric relative to default weighted extraction; red cells denote infeasible queries. Baseline ceilings and resource caps are shown in Table 2.

The per-benchmark behavior reinforces that interpretation. On `fir8`, the power objective discovers a middle point between the weighted and latency-optimal extremes. On `gemm_k8`, `gemm_blocked_k8`, and `stencil5`, the power-oriented solution returns to the baseline mapping, which is still informative because it means the extractor can represent "leave the design unchanged" as the best power choice. On `conv3x3`, all three extraction modes improve on the original graph in different ways, which indicates that the framework is not biased toward negative tradeoffs only.

Table 3. Cross-benchmark evidence for the constrained-extraction extension. Each result tuple is $A/L/P/D/U$, so the table reports total DSP and LUT use for the plotted weighted, latency-optimal, and power-optimal points.

Benchmark	Weighted A/L/P/D/U	Lat-opt A/L/P/D/U	Pwr-opt A/L/P/D/U	Pareto	Infeasible query
`fir8`	25/18/33/0/41	38/9/31/10/10	33/16/21/4/9	10	–
`dot16`	79/36/111/0/143	94/21/126/15/128	93/33/90/7/87	16	min Lat | Power cap; min Lat | LUT cap
`gemm_k8`	39/20/55/0/71	54/10/50/11/32	55/17/31/8/7	11	–
`gemm_blocked_k8`	39/14/55/0/71	62/7/38/15/0	55/11/31/8/7	8	–
`conv3x3`	16/18/18/0/20	27/9/24/10/3	18/18/15/1/12	10	–
`stencil5`	24/14/34/0/44	36/7/26/8/8	34/11/19/5/4	8	–

Q3: Do constraints reveal meaningful boundaries of the design space? Figure 2 summarizes how much the constrained queries change the answer relative to default weighted extraction. Most 'min Lat | Area cap' and 'min Lat | Power cap' queries improve their target metric substantially, which indicates that explicit budgets are active selectors rather than cosmetic post-processing. The strongest negative result is `dot16`: both 'min Lat | Power cap' and 'min Lat | LUT cap' are infeasible. This is useful evidence, not a failure of presentation. It means the extractor respects the budgets strongly enough to expose genuine impossibility under the retained frontier and rewrite set, instead of always returning a superficially different answer. **This shows that constraints reveal real boundaries of the design space and are therefore the key delta beyond SEER's equivalence-generation story.**

The resource-oriented queries also clarify what kind of alternatives the retained frontier actually contains. The zero-DSP query remains feasible across the suite, but it predictably pushes the selected points toward slower LUT-only datapaths; this is precisely the behavior one would want from a hard resource budget rather than from a soft cost penalty. Conversely, the LUT-capped query either recovers a more DSP-heavy implementation or fails outright. The contrast between those two outcomes matters because it shows that the system is not merely toggling cosmetic operator labels. It is selecting among implementations with genuinely different binding patterns and exposing when those patterns do or do not fit the requested budget.

Taken together, the three evaluation questions support a single interpretation of the project. Equality saturation is valuable because it keeps multiple arithmetic implementations alive simultaneously, but the course-level novelty here is that the same saturated graph can then be interrogated with several hardware questions. The evidence would be weaker if every constrained query simply reproduced the weighted answer, or if every profile always returned some feasible variant. Instead, the results show a richer behavior: some queries recover meaningfully better operating points, some return the baseline as the best constrained point, and some become infeasible. That is exactly the kind of extraction-level evidence needed to argue that the extension is about constrained selection rather than about generating equivalences for their own sake.

Table 4. Intel HLS component-generation validation. Analytical ranges vary across variants, but HLS estimates mostly collapse to one resource tuple per benchmark. HLS tuple is ALUT/FF/DSP/cycles for the original default-mode component.

Benchmark	Var.	Ana. DSP/LUT	HLS tuple	Unique
`conv3x3`	6	0–10 / 3–20	316/102/0/4	1
`dot16`	5	0–16 / 15–143	1009/654/24/8	1
`fir8`	6	0–11 / 2–41	317/104/0/4	1
`gemm_blk`	6	0–15 / 0–71	496/334/12/8	2
`gemm_k8`	6	0–11 / 7–71	496/334/12/8	1
`stencil5`	6	0–9 / 0–44	190/105/0/4	1

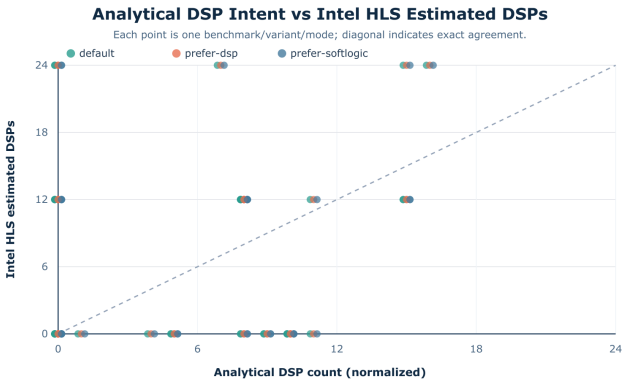

Figure 3. Analytical DSP intent versus Intel HLS estimated DSPs across all 105 component-generation reports. The weak alignment is a negative result: this HLS pass validates functional C++ generation and report extraction, but it does not preserve the analytical DSP/LUT distinctions strongly enough to replace the analytical model.

Intel HLS validation. To check whether the generated datapaths were usable outside the analytical model, the exporter emitted 35 C++ component variants with DSP- and softlogic-oriented helper calls. A deterministic C++ test harness compared every optimized variant against its original kernel on 1000 input vectors per benchmark; all variants passed. Intel HLS 21.1 then generated RTL and estimated resources for every variant under default, `prefer-dsp`, and `prefer-softlogic` modes, giving 105 HLS report rows.

Table 4 and Figure 3 show a useful limitation. Although the analytical variants span wide DSP/LUT ranges, Intel HLS usually reports identical estimated resources and schedule length for all variants of the same kernel; `gemm_blocked_k8` differs only by one ALUT in some variants. In addition, the generated `quartus.json` files report fitter resources and Fmax as `TBD`, so these results are HLS estimates rather than post-fit Quartus measurements. This strengthens the paper's scope statement: the optimizer is correct as a constrained extractor under its analytical model, while synthesis-calibrated resource binding remains future work.

7 Discussion

Relative to SEER, the project demonstrates one concrete point: equality saturation is a good way to generate HLS datapath alternatives, but constrained extraction is what makes those alternatives usable. The results are strongest where the weighted point is clearly not the right answer and where explicit budgets either recover a better implementation or expose infeasibility. That is why the negative `dot16` cases strengthen the paper rather than weaken it.

At the same time, the scope prevents broader claims. The IR is datapath-only; there is no frontend, no MLIR/CIRCT integration, and no control-flow or memory/BRAM modeling. The cost model is analytical rather than synthesis-calibrated, so the reported numbers should be interpreted as controlled relative comparisons, not as device-accurate performance or energy predictions.

The systems implication is that constrained extraction changes how an e-graph is used. In SEER, equality saturation is primarily an optimization substrate. In this project, the same saturated graph is also a queryable design-space representation: one run can answer "fastest under area budget," "smallest under latency cap," "lowest power under baseline performance," and "is this budget combination feasible?" That is the main reason the constrained interface is worth treating as its own contribution rather than as a minor variation on cost-based extraction.

Three threats to validity are worth stating explicitly. First, the bounded frontier may discard some useful candidates before the final query is answered, so the reported results characterize the retained design space rather than the full theoretical equivalence space. Second, the resource and power budgets are analytical budgets rather than synthesis-derived FPGA limits; feasibility and infeasibility therefore mean "under this model," not "for every device." Third, the power objective is intentionally simple. It distinguishes operator classes and resource bindings, but it does not model switching activity, placement, routing, or clocking effects. Those limitations matter for external fidelity, but they do not undermine the internal comparison because every extraction mode is evaluated under the same model.

8 Future Work

The next steps are direct continuations of the proposal: import kernels from C, MLIR, or CIRCT; add memory/BRAM-aware constraints; and calibrate the analytical area, latency, and power proxies against synthesis reports. These extensions would test whether the same constrained-extraction interface remains useful once costs come from a richer frontend and a physical backend rather than from a hand-authored model.

9 Conclusion

This report presented a constrained-extraction extension of the SEER idea for HLS-style arithmetic kernels. SEER established that equality saturation can generate equivalent datapaths; this project shows that explicit objectives and budgets materially change which implementation should be selected from that space. In the evaluated kernels, default weighted extraction is often not sufficient, constrained objectives recover qualitatively different operating points, and some budget combinations are genuinely infeasible. The contribution is therefore best understood as an interface result: an e-graph is more useful for HLS when it can answer constrained design questions and expose feasibility boundaries, not only return one scalar-cost expression.

References

[1] Jianyi Cheng, Samuel Coward, Lorenzo Chelini, Rafael Barbalho, and Theo Drane. 2024. SEER: Super-Optimization Explorer for High-Level Synthesis using E-graph Rewriting. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS)*. 1029–1044. https://doi.org/10.1145/3620665.3640392

[2] Jason Cong, Bin Liu, Stephen Neuendorffer, Juanjo Noguera, Kees Vissers, and Zhiru Zhang. 2011. High-Level Synthesis for FPGAs: From Prototyping to Deployment. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 30, 4 (2011), 473–491. https://doi.org/10.1109/TCAD.2011.2110592

[3] Samuel Coward, George A. Constantinides, and Theo Drane. 2022. Automatic Datapath Optimization using E-Graphs. In *2022 IEEE 29th Symposium on Computer Arithmetic (ARITH)*. 43–50. https://doi.org/10.1109/ARITH54963.2022.00016

[4] Samuel Coward, George A. Constantinides, and Theo Drane. 2023. Automating Constraint-Aware Datapath Optimization using E-Graphs. In *60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. https://doi.org/10.1109/DAC56929.2023.10247797

[5] Samuel Coward, Theo Drane, Emiliano Morini, and George A. Constantinides. 2024. Combining Power and Arithmetic Optimization via Datapath Rewriting. *Computing Research Repository (CoRR)* abs/2404.12336 (2024). https://doi.org/10.48550/arXiv.2404.12336

[6] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and Extensible Equality Saturation. *Proceedings of the ACM on Programming Languages (POPL)* 5, POPL (2021), 1–29.